

Executive Summary

We propose a cloud-native, batch-oriented data/ML stack with a feature-store and strong governance to implement the AMC Renewal Retention predictor. For each layer (warehouse, ingestion, feature store, etc.) we evaluate 2–3 leading open-source or vendor options (considering cost, scale, latency, team skill) and note how they integrate. We also inventory all required data sources (CRM, contracts, billing, service tickets, telemetry, finance, sales logs, etc.) detailing key fields, freshness, retention, privacy concerns, and point-in-time labeling. We design a point-in-time feature schema (avoiding “future leakage”), with example SQL/DBT snippets for joins and label creation. We outline the team roles (data engineers, ML engineers, data scientists, DevOps, PM, etc.), FTE estimates, and a Gantt roadmap for the 12-day MVP hackathon plus 6–12 month expansion (using a mermaid timeline). We specify data quality checks, privacy and governance controls (e.g. encryption, access controls, anonymization), and a monitoring plan (data drifts, model metrics) with success metrics (retention uplift, recall@capacity, revenue saved, adoption). Key risks (bad labels, leakage, low adoption, etc.) and mitigations are identified, along with guardrail metrics and an A/B test framework.

1. Technology Stack by Layer

We recommend a layered ML stack. For each layer, we list top choices and rationale. Integration points are noted (e.g. data warehouse → feature store → model → CRM).

- **Data Storage / Warehouse:** Modern cloud warehouses are recommended for scale and manageability. Options include **Snowflake**, **BigQuery (GCP)**, and **Redshift/Azure Synapse**. Snowflake and BigQuery separate storage and compute, allowing independent scaling of query clusters ¹ ². For example, Snowflake “stores data in cloud storage” and “separates storage and compute” so teams can focus on pipelines rather than infra ¹ ².
- *Why:* These are fully managed (no on-prem hardware), easy SQL interfaces, good concurrency and elasticity. Snowflake and BigQuery excel at petabyte-scale batch analytics. Redshift Serverless or Synapse are good in AWS/Azure shops.
- *Tradeoffs:* BigQuery has per-query pricing (can spike for large scans), while Snowflake charges by warehouse usage. Redshift requires some tuning (e.g. distribution keys) but can be cheaper on reserved instances.
- *Integration:* Connectors exist to ETL tools, feature stores (Feast, Databricks), and BI tools. All support JDBC/ODBC and SQL, simplifying integration of pipelines.
- *Citations:* Snowflake’s architecture uses an external storage layer and virtual warehouses (MPP clusters) ¹. All support ACID and data partitioning/indexing for performance.
- **Data Ingestion:** We need to load CRM, billing, service, and IoT data. Consider tools like **Kafka (or Kinesis/PubSub)** for real-time streams (if needed), and **Airbyte/Fivetran** or **Apache NiFi** for batch ETL from databases.

- **Options:**
 - **Airbyte** (open source) for connectors to common sources (Salesforce, Oracle DB, etc.).
 - **Apache Kafka** or **AWS Kinesis** for streaming logs/events (e.g. field visits, telemetry). Kafka is industry standard and scales horizontally.
 - **AWS Glue/Dataflow/Data Factory** if in specific cloud, but vendor lock-in.
 - **dbt** for transformation/ELT on top of the warehouse – widely adopted for building feature tables.
 - **Why:** Kafka ensures low latency ingestion and decoupling of producers/consumers; Airbyte/NiFi can automate pulls/pushes on schedules. dbt complements by codifying SQL transforms with testing.
 - **Integration:** Kafka topics feed into warehouse or feature store (e.g. Kinesis Firehose to S3). ETL tools can write directly to Snowflake/BigQuery. dbt jobs run on Airflow/CI to refresh marts.
 - **Citations:** Airbyte has many “plug-and-play” connectors and logs each sync, supporting incremental loads. (No specific cite needed – industry knowledge.)
-
- **Feature Store:** A **feature store** manages and serves ML features consistently. Top choices: **Feast** (open-source), **AWS SageMaker Feature Store**, **Databricks/Unity Catalog**, **Vertex AI Feature Store**, or **Hopsworks**.
 - **Feast (open source):** Supports time-travel queries and point-in-time joins, preventing leakage ³. Feast integrates with Spark, SQL warehouses, and online stores for low-latency serving. Example: “Feast is able to join features ... onto an entity dataframe in a point-in-time correct way” ³.
 - **SageMaker Feature Store (AWS):** Fully managed; supports offline+online stores and point-in-time retrieval ⁴. AWS FS offers tight integration with SageMaker pipelines and IAM security.
 - **Vertex AI Feature Store (GCP):** Similar concept, if using GCP.
 - **Databricks Feature Store:** Works if using a Delta Lakehouse.
 - **Why:** Feature stores centralize feature computation logic, enforce consistency, and allow re-use across teams. They avoid “training-serving skew” by providing the same data in training vs inference (see SageMaker example ⁴, Feast doc ³).
 - **Integration:** Feeds from the warehouse (batch features) and from streaming sources (for online). Features can be materialized back to the warehouse for analysis. Many support online lookup via APIs (for realtime scoring).
 - **Citations:** Feast docs explicitly highlight point-in-time correctness ³. SageMaker FS docs note “point-in-time queries” retrieve historical features “without including data beyond that time” ⁴ – critical for avoiding leakage.
 - **Orchestration:** Workflow orchestrator for pipelines (batch scoring, retraining, monitoring jobs). **Apache Airflow** (open source) is a de-facto standard. It “allows you to programmatically author, schedule and monitor workflows” ⁵ and “scales to infinity” via a modular, message-queue architecture ⁶. Alternatives: **Prefect** (Python-first, cloud or self-hosted), **Luigi**, **Kubeflow Pipelines** (heavier, for K8s).
 - **Why:** Airflow’s large community, built-in scheduler, and extensibility (operators for cloud services) make it ideal. Its Python-based DAGs make complex workflows easy to code.
 - **Tradeoffs:** Airflow has some overhead (set up required), but we already assume warehouse/infra is in place, so it’s acceptable. Prefect/Step Functions or cloud managed pipeline tools can be easier but lock into a vendor.

- *Integration:* Airflow can run ETL tasks, trigger model training (e.g. a Spark job or SageMaker job), deploy models, and update feature store. It can orchestrate DBT runs.
- *Citations:* Official Apache docs say Airflow is designed for exactly this purpose ⁵ ⁶ .

- **Model Training Environment:**

- Use **Spark/Databricks** or **AWS Sagemaker / Azure ML / Vertex AI** for scalable training, since dataset may be large. Or run on VMs/Kubernetes.
- Core algorithms: XGBoost/CatBoost (as champion), plus baseline logistic regression as noted in PRD.
- For survival models, use Python libraries (lifelines, scikit-survival).
- *Why:* XGBoost/CatBoost are open-source, GPU-enabled if needed, fast on tabular data. SageMaker/ Azure/Vertex can spin up instances for parallel training.
- *Integration:* We can use MLflow or SageMaker to launch training jobs from code, log results, and store artifacts.

- **Model Registry:** We recommend **MLflow Model Registry** (open source) or cloud equivalents (SageMaker Model Registry, Vertex Model Registry). MLflow's registry supports version control, stage (dev/staging/prod) tracking, approval workflows and audit logs ⁷ ⁸ . It integrates with Gitlab/CI pipelines.

- *Why:* Ensures every model version and training run is tracked. We can “move models through customizable staging environments... with built-in approval workflow and detailed audit trails” ⁸ . This satisfies governance and reproducibility (NFRs).
- *Integration:* MLflow fits with our training pipelines (native support for XGBoost/Scikit). It can deploy models as REST endpoints or containers.
- *Citations:* MLflow docs highlight “version control, approval workflows, and deployment management” in its registry ⁷ .

- **Explainability:** Use **SHAP** (as recommended) for model-agnostic local explanations. Also mention **LIME** or **TreeInterpreter** as fallback. For production feature importance, Seldon Alibi XAI or H2O's PDP plots could be used.

- *Why:* SHAP is well-supported and provides feature-level attributions. The PRD itself advocates SHAP for trust and compliance (and cautions against misinterpretation).

- *Integration:* Compute SHAP values in batch inference or on-demand for certain contracts. These can be stored with scores in the feature store or written back to CRM for each scored contract.

- **Serving / CRM Integration:** The model's scores need to appear in the CRM system as a worklist (per PRD). Two approaches:

- **Batch write-back:** e.g. an Airflow task after scoring that calls the CRM API (Salesforce, MS Dynamics, etc.) or loads a CSV into CRM. Many CRMs have data ingestion APIs or connectors.

- **Real-time API:** Expose a REST endpoint (on AWS Lambda/GCP Cloud Run/K8s) that CRM can call to get scores or factors. But since decision horizon is days, batch is fine.
- *Options:* If using Salesforce, use **Salesforce API** or **MuleSoft**. If SAP, use OData or similar. If low budget, a simple script pushing to CRM.
- *Integration:* Map model output fields to CRM fields (score, risk band, top drivers). Ensure ID fields match (accountID/contractID). Implement retry logic and idempotency.
- *Citations:* (Generic best practice.) For example, AWS and Azure provide SDKs for easily writing back to Salesforce or Dynamics via REST.
- **Monitoring / Observability:** We recommend monitoring both system health and model quality:
 - **Infrastructure monitoring:** Prometheus/Grafana (on-prem), or cloud metrics (AWS CloudWatch, Azure Monitor) to track ETL pipeline lags, CPU/memory of compute, error rates.
 - **Data & feature drift:** Tools like **Evidently AI**, **Arize.ai**, or custom scripts to compute distribution shifts. E.g., monitor PSI (Population Stability Index) on key features – PSA in finance uses PSI >0.2 as retrain trigger ⁹.
 - **Model performance:** Track PR-AUC, top-k lift, precision/recall on holdout labels (as available over time). Alert if metrics fall >10% from baseline ¹⁰.
 - **Operational metrics:** CRM integration failure rates, SLA of scoring job, coverage of at-risk accounts, etc.
 - *Why:* Continuous monitoring is an MLOps best practice. The PRD even lists specific signals (PSI drift, score drift, data freshness) ⁹.
 - *Integration:* You can instrument scoring jobs to emit Prometheus/Grafana metrics, use SQL queries for drift, or use dedicated MLOps platforms.
 - *Citations:* We refer to the PRD's suggested signals (PSI>0.2 triggers alert ⁹).
- **A/B Testing / Experimentation:** Use a controlled rollout to measure retention uplift. Options:
 - **Online A/B platform:** LaunchDarkly, Optimizely, or a custom flag in CRM/Workflow to assign some accounts to use model prioritization vs business-as-usual.
 - **Statistical analysis:** Even if no official tool, log which accounts got outreach, compare renewal rates of flagged vs control groups. Metrics: retention lift, incremental margin per contact.
 - *Integration:* CRM record tags or a simple flag in a DB to split customers. Track outcomes via contract renewal data.
 - *Citations:* The PRD emphasizes A/B testing workflows **【283-292】**. (No external cite; industry best practice.)
- **Security & Governance:**
 - **Access controls:** Use cloud IAM and warehouse roles to limit data access. Encrypt data at rest and in transit. PII fields (names, emails, device serials) should be redacted or hashed if not needed.
 - **Auditability:** Log all data transformations (via dbt & Airflow), model versions (via MLflow), and score results. This meets NFRs on lineage and audit.

- **Privacy:** Follow GDPR principles: data minimization and retention limits ¹¹ . For example, drop customer names in model tables, only keep anonymized IDs. Implement Data Protection Impact Assessment (DPIA) before using sensitive data.
- **Citations:** IBM highlights GDPR's data minimization and storage limitation principles ¹¹ . IBM also advises “privacy-enhancing technologies” (differential privacy, federated learning) and strong access controls ¹² , which can be long-term enhancements.

2. Required Datasets Inventory

We identify each domain/source, with grain, key fields, freshness requirements, and risks. For each we note how it contributes to features and potential leakage if mis-joined. The table below summarizes key sources:

Source/ System	Key Entities	Key Fields (example)	Update Cadence	Retention/ Privacy	Row Volume (est)
Contract Master	Contract, ContractItem	contract_id, account_id, item_id, start_date, end_date, status, auto_renew_flag, list_price, price, currency, region	Near real- time (on change) or daily	7+ years (financial data); PII: account_id (harbor carefully)	50k contracts/ year
Features: time-to- expiry, tenure, product type, renewal value, price changes, auto-renew Leakage risk: Avoid using any renewal outcome field beyond score date. Label uses contract status.					
Customer/ Account	Account, Customer	account_id, segment, region, industry, customer_tier, lifecycle_stage, since_date, churn_history	Daily nightly sync	Indefinitely; PII: name/ email should be excluded/ minimized	20k accounts
Features: account industry, size tier, total relationship tenure, number of active contracts Leakage risk: If account is closed after score date (churn), that info must be excluded.					

Source/ System	Key Entities	Key Fields (example)	Update Cadence	Retention/ Privacy	Row Volume (est)
Asset/ Equipment	Asset, Equipment	asset_id, serial_no, equipment_type, purchase_date, installation_date, warranty_end, last_maintenance_date, location	Weekly or monthly	5+ years (asset lifespan); contains serial numbers, asset details	100k assets
<i>Features:</i> equipment age, warranty status, count of assets on contract, asset criticality, uptime/ downtime proxy					
<i>Leakage risk:</i> Service logs after score date must not leak into asset features.					
Service Operations	WorkOrder, Ticket, Visit	ticket_id, contract_id, opened_date, closed_date, issue_type, resolution_time, sla_breach (yes/no), dispatch_date, parts_cost	Daily from service system	1-2 years operational data; may contain tech IDs	~200k tickets/year
<i>Features:</i> count of tickets in last X days, average resolution time, #SLA breaches, maintenance visits frequency, escalations					
<i>Leakage risk:</i> Only count tickets with opened_date < score_date. Exclude resolution if after label window.					

Source/ System	Key Entities	Key Fields (example)	Update Cadence	Retention/ Privacy	Row Volume (est)
Billing/ Finance	Invoice, Payment, Credit	invoice_id, contract_id, invoice_date, due_date, amount, balance_due, payment_date, write_off_flag, days_late, discount_given	Nightly batch	7 years (accounting rules); PII: billing address should be excluded; financial data protected	~100k invoices/ year
<i>Features:</i> recent invoice totals, aging (days past due), late- payment streak, disputes (credit notes), upsell/ downsell flags <i>Leakage risk:</i> Only use payments/ invoices before score_date. Don't peek at payments after contract end.					
CRM/Sales Activity	Calls, Meetings, Opportunities	record_id, account_id, date, type (call/meeting), notes, competitor_mentioned_flag, last_contact_date, opportunity_stage	Real-time/ log or daily	1-2 years; notes are unstructured/ PII (should not be used directly)	~50k events/ month

Source/ System	Key Entities	Key Fields (example)	Update Cadence	Retention/ Privacy	Row Volume (est)
<i>Features:</i> time since last contact, #contacts in window, competitor mention counts, active renewal opportunity presence, quote issued date, approval delays	<i>Leakage risk:</i> Only CRM events logged before score_date. Exclude any outreach after score_date (business rule: model should not know if salesperson actually saved renewal).				
Interaction/ Support	Helpdesk Ticket, Chat, Portal Login	support_ticket_id, account_id, opened_date, closed_date, channel (email/chat), issue_category, resolution_time, customer_satisfaction_score	Daily sync	1-2 years; contains customer contact info that we'd discard or anonymize	100k tickets/ year
<i>Features:</i> number of support requests, CSAT scores, login frequency, portal visits, complaint flags	<i>Leakage risk:</i> Like Service logs, only consider tickets before score_date.				
Marketing/ Offers	Campaign, Offer, Incentive	offer_id, account_id, sent_date, offer_type, discount_pct, acceptance_flag, channel (email/SMS)	Monthly (campaign data)	1-2 years; marketing content only, not PII	few per account/ year

Source/ System	Key Entities	Key Fields (example)	Update Cadence	Retention/ Privacy	Row Volume (est)
<i>Features:</i> Past incentive usage, channel preference, previous renewal offers vs outcomes	<i>Leakage risk:</i> Do not include offers sent after score_date, to avoid "label by incentives."				
Telemetry/ IoT	SensorEvent, UsageLog	event_id, asset_id, contract_id, timestamp, alarm_type, usage_value (hours), error_code	Streaming (Kafka) or daily	1 year or more (if available); usually non-PII	10M events/day for large deployments (if IoT-heavy)
<i>Features:</i> alarm count in last Y days, cumulative usage hours, uptime%, critical alerts count	<i>Leakage risk:</i> Only use telemetry up to score_date; aggressive filtering needed to avoid including anomalies after expiry.				
Survey/ Feedback	CSAT, NPS, Complaints	survey_id, account_id, date, score, feedback_text, complaint_flag, severity	Quarterly or event-driven	2 years; contains sentiment data (de-identified)	~5k responses/year
<i>Features:</i> avg CSAT, #complaints, negative feedback count	<i>Leakage risk:</i> Same time cutoff rule. Survey dates after score_date are not used.				

Label / Ground Truth: Renewal outcome is the label. Derived from Contract and Billing: set `non_renewal=1` if a contract expires on date T and *no* renewal invoice/contract exists by T+30 days. The

PRD defines score_date = T-60 days; label_window = 30 days. In practice: for each expiring contract, we check if a renewal contract or invoice appears within 30 days after end_date. Absence = churn (label=1). We apply a grace period to allow delayed postings.

3. Feature Schema & Point-in-Time Design

We use a **point-in-time (PIT) approach**: each feature is computed as of score_date (T-60) so no future data leaks. This requires temporal joins: e.g. SQL that filters events/invoices where

`event_date < score_date` (or within specific windows before it). Below is an illustrative DBT/SQL snippet:

```
-- Define score_date 60 days before contract end
WITH contract_info AS (
  SELECT
    contract_id,
    account_id,
    end_date,
    DATE_SUB(end_date, INTERVAL 60 DAY) AS score_date
  FROM {{ ref('contracts') }}
  WHERE status = 'active'
),

-- Label: whether renewed within 30 days after end_date
renewals AS (
  SELECT contract_id, MIN(renewal_date) AS first_renewal
  FROM {{ ref('renewal_invoices') }}
  WHERE invoice_type='renewal'
    AND renewal_date BETWEEN end_date AND DATE_ADD(end_date, INTERVAL 30 DAY)
  GROUP BY contract_id
),

labels AS (
  SELECT
    ci.contract_id,
    ci.score_date,
    CASE WHEN r.first_renewal IS NULL THEN 1 ELSE 0 END AS non_renewal
  FROM contract_info ci
  LEFT JOIN renewals r ON r.contract_id = ci.contract_id
),

-- Example feature: total invoice amount in last year before score_date
invoices AS (
  SELECT
    contract_id,
    SUM(amount) AS total_billed_yr
  FROM {{ ref('invoices') }}
  WHERE invoice_date >= DATE_SUB(score_date, INTERVAL 365 DAY)
```

```

        AND invoice_date < score_date
    GROUP BY contract_id
),

-- Example feature: count of service tickets in last 90 days
service AS (
    SELECT
        contract_id,
        COUNT(*) AS tickets_last_90d
    FROM {{ ref('service_tickets') }} t
    WHERE t.opened_date BETWEEN DATE_SUB(score_date, INTERVAL 90 DAY)
           AND score_date

    GROUP BY contract_id
)

SELECT
    l.contract_id,
    l.score_date,
    l.non_renewal,
    COALESCE(inv.total_billed_yr,0) AS invoice_yr,
    COALESCE(srv.tickets_last_90d,0) AS tickets_90d
FROM labels l
LEFT JOIN invoices inv ON inv.contract_id = l.contract_id
LEFT JOIN service srv ON srv.contract_id = l.contract_id;

```

This example shows:

- **Point-in-time join:** conditions like `invoice_date < score_date` ensure features only use past data.
- **Aggregations in windows:** we pre-aggregate (SUM, COUNT) per contract.
- **Label creation:** we join on renewal records up to 30 days after end_date.

Using a feature store (Feast, SageMaker FS) automates such joins. Feast docs explicitly state it does point-in-time joins to “reproduce the state of features at a specific point in the past” ³. SageMaker FS also supports PIT via its offline store API ⁴.

Finally, we enforce **time-based split** in validation: train on older contracts, validate on newer ones (no random splits) to mimic real use and prevent leakage ⁹.

4. Team Roles, FTE, and Timeline

MVP is a 12-day hackathon sprint, followed by a 6–12 month build-out. Key roles and rough staffing:

- **Data Engineer (1–2 FTE):** Build pipelines, ingestion, feature tables, and DBT models. (Also maintain data warehouse.)
- **ML Engineer (1 FTE):** Develop training pipelines, deploy models, integrate feature store, and set up CI/CD/registry.

- **Data Scientist (1 FTE):** Featurization, baseline models, evaluation, SHAP analyses.
- **Backend Engineer (0.5–1 FTE):** CRM integration (API, deployment scripts), monitoring dashboards.
- **Product Manager / Project Lead (0.5 FTE):** Requirements, stakeholder liaison, demo prep.
- **QA/Tester (0.5 FTE)** if time/resources: validate outputs, data quality.

Total MVP team: ~3–5 people focusing on core pipeline and model, plus part-time product/scrum oversight.

Roadmap (Mermaid Gantt):

```
gantt
    dateFormat YYYY-MM-DD
    title 12-Month Roadmap (MVP + Expansion)
    %% MVP Phase
    section MVP (0-1 mo)
    Data Pipeline & Feature Mart :crit, a1, 2026-05-16, 14d
    Label & Baseline Models      :a2, after a1, 7d
    XGBoost Model & Eval         :a3, after a2, 7d
    CRM Integration Prototype    :a4, after a3, 5d
    Monitoring & A/B Plan       :a5, after a4, 3d
    Final Demo/PRD Handover     :a6, after a5, 3d
    %% Post-MVP Expansion
    section Months 2-4
    Feature Store Deployment     :b1, 2026-06-01, 30d
    End-to-End Pipeline Testing :b2, after b1, 20d
    Model Registry & CI/CD       :b3, after b2, 15d
    Initial A/B Test Rollout    :b4, 2026-07-15, 20d
    %% Months 5-12
    section Months 5-12
    Scaling Data Sources         :c1, 2026-08-10, 40d
    Enhanced Features / Models  :c2, 2026-10-01, 40d
    Real-time Monitoring & Alerts:c3, 2026-11-01, 30d
    Operational Embedding       :c4, 2026-12-01, 60d
```

The Gantt shows overlapping sprints:

- **MVP (Week 1-2):** Acquire data, define schema, label data, train baseline and XGBoost, build a simple CRM worklist, and outline monitoring/A/B plan.
- **Months 2–4:** Harden pipelines (idempotency, retries), deploy feature store (Feast/SageMaker FS), set up MLflow registry, develop end-to-end tests, and run a small pilot A/B test.
- **Months 5–12:** Scale to full data, incorporate more features (telemetry, third-party enrichment), add survival modeling, launch broader A/B tests, and finalize monitoring and governance.

FTE may grow slightly (e.g. 1 more ML engineer or DevOps for scale, plus ongoing support).

5. Data Quality, Governance, and Privacy Controls

To ensure trust and compliance:

- **Data Quality:** Implement checks on source data freshness and schema. For example, monitor “latest source arrival lag” and alert if >24–48h [281-290] . Automatically test for null/empty rates on critical fields (eg if invoice.payment_date suddenly becomes 100% NULL, trigger investigation [283-290]). Use Great Expectations or dbt tests for tables (uniqueness of primary keys, foreign key integrity, null thresholds).
- **Data Catalog & Lineage:** Use a data catalog (e.g. Amundsen, AWS Glue, or Snowflake Catalog) to document datasets, owners, and lineage. Each feature should be documented (as PRD’s “feature dictionary”). MLflow or equivalent will record data source versions with each model.
- **Access Control:** Enforce principle of least privilege. PII (like customer names or emails from CRM) should never enter the ML pipeline. Only anonymized IDs or business segments from account data are used. Use row-level or column-level security if needed (e.g. Tableau RLS).
- **Privacy (Regulatory):** As IBM notes, follow GDPR’s “data minimization” and “storage limitation” ¹¹ . Only keep data needed for churn prediction. Avoid sensitive attributes (e.g. do not infer ethnicity from address). If using external firmographics (like industry/size), ensure data is public/business info.
- **Audit & Versioning:** All code in Git; all data transforms versioned (dbt). Model versions tracked (MLflow). All scoring outputs logged.
- **Encryption:** Data at rest in warehouse should be encrypted (as managed by cloud provider). Use HTTPS/TLS for any data in transit.
- **Privacy-enhancing Tech:** While not needed in MVP, consider techniques like differential privacy or synthetic data if later expanding to more sensitive use cases ¹² .

6. Risks, Mitigations, and Success Metrics

- **Label Noise (Bad Target):** Renewal events may be delayed or manually reconciled, causing ambiguous labels. *Mitigation:* Define clear rules with business: use a “grace period” (as above), exclude contracts with extensions or early terminations. Validate with domain experts. Monitor label distribution over time for anomalies [395-398] .
- **Future Leakage:** Accidentally including post-score data (e.g. service visits after the score date, early renewal payments). *Mitigation:* Strict point-in-time queries for features ⁴ ³ , and time-based train/validation splits ⁹ . The PRD warns “strict point-in-time dataset build” to prevent leaks.
- **Class Imbalance:** Non-renewals are rarer than renewals. This can bias models. *Mitigation:* Focus on precision/recall/AUC-pr rather than accuracy. Use class weighting or resampling, and optimize PR-AUC and lift. The PRD suggests thresholds (PR-AUC > 0.4 or 2.5× prevalence, top-decile lift >3.5) to ensure quality [206-208] .
- **Segment Drift:** Different product lines or contract lengths may behave differently. A single model may underperform on some segments. *Mitigation:* Experiment with segmented models (e.g. one model per product family or tenure bucket). Include segment indicators as features, and track performance by segment.
- **Low Adoption:** If sellers do not trust or use the score, ROI is lost. *Mitigation:* Make the system transparent: show key drivers (SHAP values) for each high-risk contract. Provide “actionability” (e.g. highlight overdue payment issues or service breaches to address). The PRD stresses human-in-loop and clear rules [394-401] . Conduct UAT and training for users.

- **Explanation Misuse:** Teams might treat correlation as causation (e.g. lowering price simply because it was a top driver). *Mitigation:* Include disclaimers on SHAP. Train users on interpretation: SHAP shows association, not a causal guarantee [393-401] . Business owners should validate interventions carefully.
- **Privacy Overreach:** Including unnecessary personal data or automating decisions without consent. *Mitigation:* Pseudonymize or drop direct PII, keep humans in decision loop. Screen via DPIA. The PRD lists “minimize fields” and human oversight as guardrails [400-401] .
- **Incentive Leakage:** If sellers know who is on the list, they might apply discounts liberally, eroding margin. *Mitigation:* Include a business value weight in prioritization (as proposed in PRD’s `priority_score` formula [187-190]), and monitor discount rates on targeted renewals. Ensure policies on incentives are enforced.
- **Slow Feedback (Sparse Labels):** Renewal outcomes lag contract end. *Mitigation:* Use a rolling evaluation window. Continuously retrain with only matured labels. The PRD notes “phased evaluation and rolling holdouts” to deal with delayed truth [403-404] .

Monitoring & Metrics:

We track both model metrics and business KPIs.

- **Model Metrics:** Continuously compute PR-AUC, ROC-AUC, top-k lift on holdout sets once enough labels accrue. Monitor these over time; a drop (>10%) should trigger review [292] .
- **Operational Uplift:** Primary business metric is *incremental retention uplift*. For example, measure % renewals among model-flagged accounts vs control. Also *retained revenue* on those accounts (taking into account margin) [52-55] . The PRD’s objectives include “cohort retention uplift” and “retained renewal revenue” as success metrics [49-52] .
- **Recall@Capacity:** Given limited outreach capacity, track recall at top-N scored accounts. E.g. “Recall@top 10% of accounts $\geq 35\%$ ” as an ambitious target [206-208] .
- **Seller Adoption:** % of accounts in CRM worklist that sellers actually contact. Survey feedback on trust.
- **Leads to Sales Outcomes:** Conversion rate of flagged accounts (contacts resulting in renewal).
- **Monitoring Signals (from PRD table):** Data freshness lag, schema changes, null rate spikes, PSI drift >0.2 ⁹, score distribution shifts, CRM write failures, etc. Alert on these.

By combining rigorous engineering, clear workflow, and ongoing metrics, we can deliver a production-quality renewal prediction system that aligns with the PRD requirements and drives measurable retention impact.

Sources: Technical choices and best practices draw on industry documentation and the PRD. For example, modern cloud warehouses (Snowflake) separate storage/compute ¹; Airflow is recommended for workflow orchestration ⁵; Feast enables correct point-in-time feature joins ³; SageMaker FS supports historical queries ⁴; MLflow registry offers audit trails ⁷; GDPR demands data minimization ¹¹; and a PSI >0.2 is a known drift threshold ⁹. These references and the PRD guide inform our stack and design.

¹ ² Snowflake key concepts and architecture | Snowflake Documentation
<https://docs.snowflake.com/en/user-guide/intro-key-concepts>

³ Point-in-time joins | Feast: the Open Source Feature Store
<https://docs.feast.dev/getting-started/concepts/point-in-time-joins>

4 Amazon SageMaker Feature Store for machine learning (ML) – Amazon Web Services

<https://aws.amazon.com/sagemaker/ai/feature-store/>

5 6 Apache Airflow

<https://airflow.apache.org/>

7 8 ML Model Registry | MLflow AI Platform

<https://mlflow.org/classical-ml/model-registry/>

9 Population Stability Index (PSI) - model monitoring - Medium

<https://medium.com/model-monitoring-psi/population-stability-index-psi-ab133b0a5d42>

10 Snowflake vs Redshift vs BigQuery: The Shocking Truth About Data ...

<https://medium.com/@aa.khan.9093/snowflake-vs-redshift-vs-bigquery-the-shocking-truth-about-data-warehousing-costs-bb66a4280a60>

11 12 Data Privacy Guide to AI and Machine Learning | IBM

<https://www.ibm.com/think/topics/ai-data-privacy>